

COL331 A3

Sarthak Panda (2022CS51217)

April 2025

1 Introduction

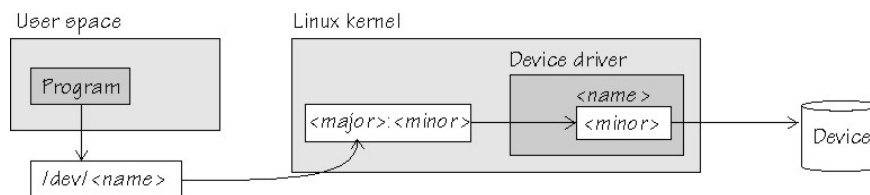
The current implementation of module has the following file organization.

```
SMART_BLOCK
├-> smart_block.c
├-> Makefile
├-> Basic.sh (Testcase)
├-> Intermediate.sh (Testcase)
├-> Advanced.sh (Testcase)
```

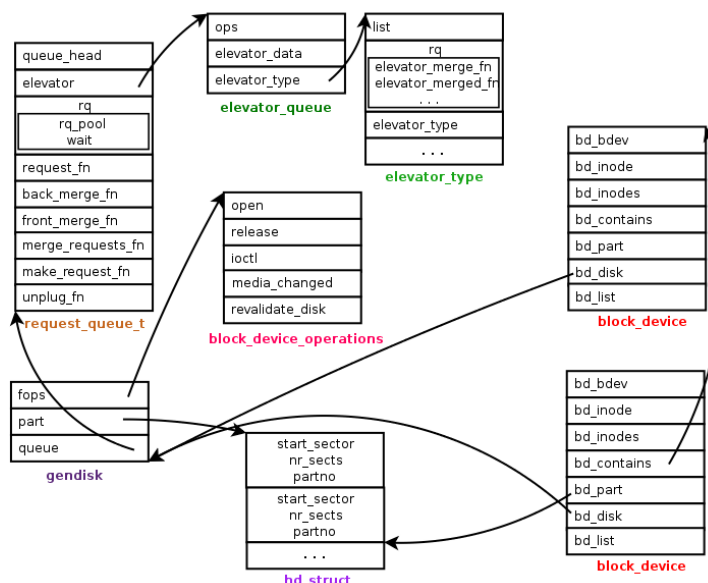
2 Driver Functionality

2.1 General Overview over Block Drivers

The first to understand the block drivers is to understand the major and minor numbers. The below figure makes it a bit clear what major and minor number refer to. For example in `/dev/sda0` and `/dev/sda1`, both of them refer to same major number with different minor numbers.



To get a major number for our device we call `register_blkdev` inside our module init. The below image (from Resource Link [1]) give us a general idea of the workflow we would need to follow. The three main components are `block_device`, `gendisk` and `request_queue`. Rest details of each we would see as we go to subsequent sections.



2.2 Device Structure

```
1 struct smart_block_dev
2 {
3     //Backing File representing disk
4     unsigned long size;
5     struct file *storage_file;
6
7     //Cache related Parameters
8     atomic_long_t cache_size;
9     atomic_long_t write_count;
10    spinlock_t lock;
11    struct list_head cache_list;
12    unsigned long max_cache_size;
13
14    //Parameters to facilitate Asynchronous writes
15    struct workqueue_struct *wq;
16    struct work_struct flush_work;
17    struct list_head pending_writes;
18    spinlock_t pending_lock;
19 };
```

Listing 1: Kernel code snippet from hello/hello.h

To start with the driver code we first start by defining a custom device structure. As shown our device structure mainly contains three major parts one part of managing Backing file which will act as a disk, The second part is related for management of cache structures and finally a third part for asynchronous writes from the cache to the disk. The important thing to note is maintain pointer to head of linked lists of both cache and pending tasks.

2.3 Cache Structure & Pending Task Structures

```
1 struct cache_entry
2 {
3     sector_t sector;
4     unsigned int num_sectors;
5     u8 *data;
6     struct list_head list;
7 };
```

Listing 2: Old Kernel code for open syscall in fs/open.c

```
1 struct pending_work
2 {
3     struct list_head list;
4     struct cache_entry *entry;
5 };
```

Listing 3: New Kernel code for open syscall in fs/open.c

For both cache and pending task we use a linked list type data structure. Only thing to note is when cache is set to be flushed all cache entries get removed from cache list and added to pending work list.

2.4 Module Initialization

In the `my_block_init()` function, we first register a block-device major number with `register_blkdev()` and allocate a `smart_block_dev` structure. We then initialize its internal lists, workqueue, spinlocks and atomic counters, and open the backing storage file. Next, we allocate and configure a `blk_mq_tag_set` for multi-queue support, specifying queue depth, flags and driver data, and call `blk_mq_alloc_tag_set()`. We obtain the request queue from a newly allocated `gendisk`, initialize it with `blk_mq_init_allocated_queue()`, set non-rotational and write-cache flags, and define the logical block size. Finally, we populate the `gendisk` fields (major, minors, name, `fops`, queue and private data), set its capacity and call `add_disk()` to register the device, completing the initialization.

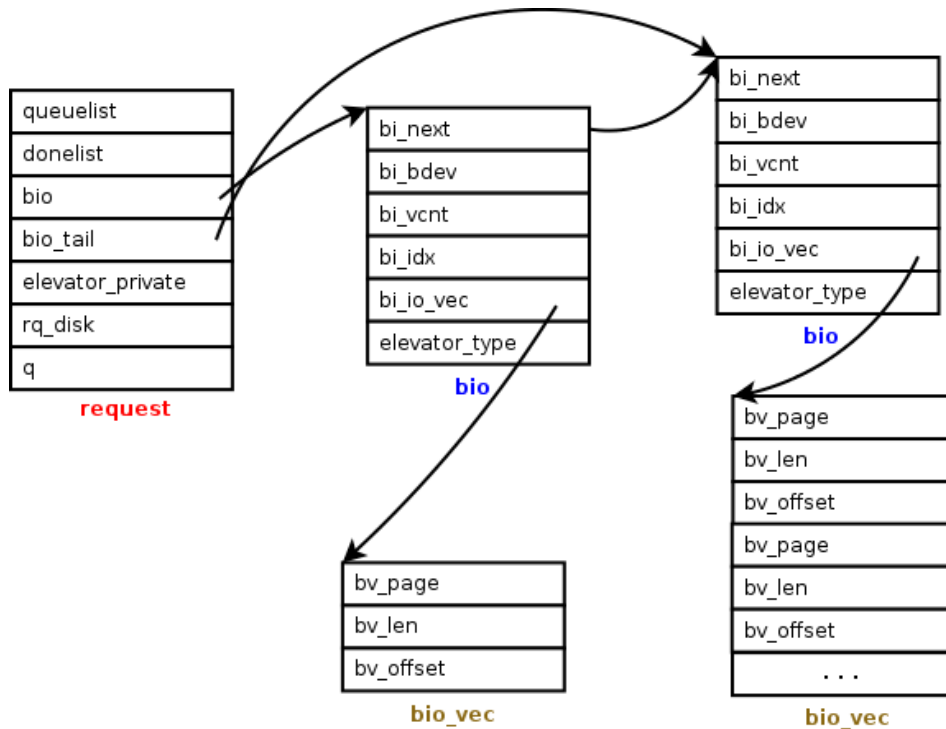
```

1 static __init int my_block_init(void){
2     int ret,ret1;
3     struct blk_mq_tag_set *tag_set;
4     //get major num
5     major_num = register_blkdev(MY_BLOCK_MAJOR_TRIAL, MY_BLKDEV_NAME);
6
7     //allocate and intialize your custom device structure
8     smart_block_device = kzalloc(sizeof(struct smart_block_dev), GFP_KERNEL);
9     INIT_LIST_HEAD(&smart_block_device->cache_list);
10    INIT_WORK(&smart_block_device->flush_work, flush_work_fn);
11    smart_block_device->wq=alloc_workqueue("smart_block_flush", WQ_MEM_RECLAIM | WQ_HIGHPRI, 1);
12    INIT_LIST_HEAD(&smart_block_device->pending_writes);
13    spin_lock_init(&smart_block_device->pending_lock);
14    smart_block_device->size = MYBLK_SIZE_SECT;
15    smart_block_device->storage_file = filp_open(backing_file_path, O_RDWR | O_CREAT, 0600);
16    spin_lock_init(&smart_block_device->lock);
17    atomic_long_set(&smart_block_device->cache_size, 0);
18    atomic_long_set(&smart_block_device->write_count, 0);
19    smart_block_device->max_cache_size=cache_size_bytes;
20
21    //allocate and intialize tag set
22    tag_set = kzalloc(sizeof(*tag_set), GFP_KERNEL);
23    tag_set->ops = &smart_block_mq_ops;
24    tag_set->nr_hw_queues = 1;
25    tag_set->queue_depth = 128;
26    tag_set->numa_node = NUMA_NO_NODE;
27    tag_set->cmd_size = 0;
28    tag_set->flags = BLK_MQ_F_SHOULD_MERGE | BLK_MQ_F_BLOCKING;
29    tag_set->driver_data = smart_block_device;
30    ret = blk_mq_alloc_tag_set(tag_set);
31
32    //allocate gendisk structure
33    smart_block_disk = blk_alloc_disk(NUMA_NO_NODE);
34    smart_block_req_qu = smart_block_disk->queue;
35
36    //intialize and setup request queue
37    ret = blk_mq_init_allocated_queue(tag_set, smart_block_req_qu);//we are using multiqueue
38    blk_queue_flag_set(QUEUE_FLAG_NONROT, smart_block_req_qu);
39    blk_queue_logical_block_size(smart_block_req_qu, KERNEL_SECTOR_SIZE);
40    blk_queue_write_cache(smart_block_req_qu, true, false);//2nd sets write back to true
41
42    //intialize the gendisk
43    smart_block_disk->major = major_num;
44    smart_block_disk->first_minor = DEVICE_MINOR_NUM_FIRST;
45    smart_block_disk->minors = DEVICE_MINOR_NUMS_MAX;
46    strcpy(smart_block_disk->disk_name, MY_BLKDEV_NAME);
47    smart_block_disk->fops = &smart_block_fops;
48    smart_block_disk->queue = smart_block_req_qu;
49    smart_block_disk->private_data = smart_block_device;
50    set_capacity(smart_block_disk, MYBLK_SIZE_SECT);
51    ret1 = add_disk(smart_block_disk);
52    printk(KERN_INFO "smart_block: Initialized successfully\n");
53    return 0;
54 }
55
56 module_init(my_block_init);

```

Listing 4: Kernel code snippet from hello/hello.h

2.5 Implementation of Request-queue



Let us first understand request queue. The role of request queue is to facilitate the read and writes to device. Each request in request queue points to a `bio_vec` structure which contains information about page, length and offset of data to be read or to be written. Below is the code that exactly how extract the information from request and ultimately calling a internal function `smart_block_transfer` which will check if it read/write request and handle it. We will this internal function's detail in next section.

```

1 static struct blk_mq_ops smart_block_mq_ops = {
2     .queue_rq = smart_block_queue_rq,
3 };
4
5 static blk_status_t smart_block_queue_rq(struct blk_mq_hw_ctx *hctx, const struct
6     blk_mq_queue_data *bd) {
7     struct request *rq = bd->rq;
8     struct smart_block_dev *dev = smart_block_device;
9     struct req_iterator iter;
10    struct bio_vec bvec;
11    sector_t pos = blk_rq_pos(rq);
12    blk_status_t ret = BLK_STS_OK;
13    blk_mq_start_request(rq);
14    rq_for_each_segment(bvec, rq, iter) {
15        unsigned int len = bvec.bv_len;
16        void *buffer = page_address(bvec.bv_page) + bvec.bv_offset;
17        int dir = rq_data_dir(rq);
18        smart_block_transfer(dev, pos, len / KERNEL_SECTOR_SIZE, buffer, dir);
19        pos += len / KERNEL_SECTOR_SIZE;
20    }
21    done:
22    blk_mq_end_request(rq, ret);
23    return BLK_STS_OK;
24 }

```

Listing 5: Kernel code snippet from hello/hello.h

2.5.1 Implementation details of Read

```
1 static void smart_block_transfer(struct smart_block_dev *dev, sector_t sector, unsigned int nsect
2 , char *buffer, int write) {
3     unsigned long offset = sector * KERNEL_SECTOR_SIZE;
4     unsigned long nbytes = nsect * KERNEL_SECTOR_SIZE;
5     unsigned long flags;
6     if (write) { //We will see in next section
7     else {
8         unsigned char *sectors_handled;
9         struct cache_entry *entry;
10        unsigned long flags;
11        loff_t pos = offset;
12        //Read from Disk
13        kernel_read(dev->storage_file, buffer, nbytes, &pos);
14        //Overlay latest changes from cache, without flushing to disk
15        sectors_handled = kzalloc(nsect, GFP_ATOMIC);
16        spin_lock_irqsave(&dev->lock, flags);
17        list_for_each_entry_reverse(entry, &dev->cache_list, list) {
18            sector_t entry_start = entry->sector;
19            sector_t entry_end = entry_start + entry->num_sectors;
20            sector_t req_end = sector + nsect;
21            sector_t overlap_start, overlap_end;
22            if (entry_end <= sector || entry_start >= req_end) continue;
23            overlap_start = max(entry_start, sector);
24            overlap_end = min(entry_end, req_end);
25            for (sector_t s = overlap_start; s < overlap_end; s++) {
26                unsigned idx = s - sector;
27                if (!sectors_handled[idx]) {
28                    unsigned long buf_offset = idx * KERNEL_SECTOR_SIZE;
29                    unsigned long cache_offset = (s - entry_start) * KERNEL_SECTOR_SIZE;
30                    memcpy(buffer + buf_offset, entry->data + cache_offset, KERNEL_SECTOR_SIZE);
31                    sectors_handled[idx] = 1;
32                }
33            }
34        }
35        spin_unlock_irqrestore(&dev->lock, flags);
36        kfree(sectors_handled);
37    }
38 }
```

Listing 6: Kernel code snippet from hello/hello.h

In the read path of `smart_block_transfer()`, we compute the byte offset and length from the requested `sector` and `nsect`, then perform a bulk read from the backing file via `kernel_read()`. To ensure the most recent writes are visible without flushing, we allocate a temporary bitmap `sectors_handled` and acquire the device spinlock. We traverse the `cache_list` in reverse to apply any cached cache entries that overlap the request range: for each overlapping sector not yet handled, we copy the fresh data from the cache entry into the user buffer and mark it as served. After processing all cache hits, we release the lock, free the bitmap, and return, delivering a composite view that merges on-disk data with in-flight cached writes.

2.5.2 Implementation details of Write

In the write branch of `smart_block_transfer()`, we first allocate a new `cache_entry` and its data buffer using `kmalloc()`, copying `nbytes` from the incoming `buffer` into `entry->data`. We record the target `sector` and `num_sectors` in the entry, then acquire the device spinlock to safely append the entry to `dev->cache_list` and atomically increase `dev->cache_size`. If the accumulated cache exceeds `dev->max_cache_size`, we trigger an immediate flush via `__flush_cache_unsafe()` and schedule deferred writeback on the workqueue with `queue_work()`. Finally, we release the lock, leaving the new write cached in memory and queued for eventual persistence.

```

1  if (write) {//Continuing from above where we left
2      struct cache_entry *entry = kmalloc(sizeof(*entry), GFP_ATOMIC);
3      if (!entry) {
4          printk(KERN_ERR "smart_block: Cache entry alloc failed\n");
5          return;
6      }
7      entry->data = kmalloc(nbytes, GFP_ATOMIC);
8      if (!entry->data) {
9          kfree(entry);
10         printk(KERN_ERR "smart_block: Data alloc failed\n");
11         return;
12     }
13     memcpy(entry->data, buffer, nbytes);
14     entry->sector = sector;
15     entry->num_sectors = nsect;
16
17     spin_lock_irqsave(&dev->lock, flags);
18     list_add_tail(&entry->list, &dev->cache_list);
19     atomic_long_add(nbytes, &dev->cache_size);
20
21     if (atomic_long_read(&dev->cache_size) >= dev->max_cache_size) {
22         __flush_cache_unsafe(dev);
23         queue_work(dev->wq, &dev->flush_work);
24     }
25     spin_unlock_irqrestore(&dev->lock, flags);
26 }

```

Listing 7: Kernel code snippet from hello/hello.h

2.6 Flush cache and Asynchronous writes to disk

```

1  static void flush_work_fn(struct work_struct *work) {
2      struct smart_block_dev *dev = container_of(work, struct smart_block_dev, flush_work);
3      struct pending_work *pwork, *tmp;
4      struct cache_entry *entry;
5      unsigned long flags;
6      loff_t pos;
7      spin_lock_irqsave(&dev->pending_lock, flags);
8      list_for_each_entry_safe(pwork, tmp, &dev->pending_writes, list) {
9          list_del(&pwork->list);
10         spin_unlock_irqrestore(&dev->pending_lock, flags);
11         entry = pwork->entry;
12         pos = entry->sector * KERNEL_SECTOR_SIZE;
13         kernel_write(dev->storage_file, entry->data, entry->num_sectors * KERNEL_SECTOR_SIZE, &
14             pos);
15         atomic_long_inc(&dev->write_count);
16         kfree(entry->data); kfree(entry); kfree(pwork);
17         spin_lock_irqsave(&dev->pending_lock, flags);
18     }
19     spin_unlock_irqrestore(&dev->pending_lock, flags);
20 }

```

Listing 8: Kernel code snippet from hello/hello.h

The `flush_cache()` helper acquires the device lock and invokes `__flush_cache_unsafe()` to detach all `cache_entry` objects from `dev->cache_list`, decrementing `dev->cache_size` accordingly and packaging each entry into a `pending_work` node, which is enqueued under `dev->pending_writes` while holding `pending_lock`; entries for which allocation of a `pending_work` fails are synchronously written back via `kernel_write()`. After unlocking, `flush_cache()` schedules `flush_work_fn()` on the workqueue, which in turn dequeues each `pending_work`, writes its payload to the backing file, increments the write-counter, and frees both the cache entry and work node, under proper lock protection to ensure safe, asynchronous persistence of cached writes.

```

1 static void __flush_cache_unsafe(struct smart_block_dev *dev) {
2     struct cache_entry *entry, *tmp;
3     struct pending_work *work;
4     list_for_each_entry_safe(entry, tmp, &dev->cache_list, list) {
5         list_del(&entry->list);
6         atomic_long_sub(entry->num_sectors * KERNEL_SECTOR_SIZE, &dev->cache_size);
7         work = kmalloc(sizeof(*work), GFP_ATOMIC);
8         if (work) {
9             unsigned long flags;
10            INIT_LIST_HEAD(&work->list);
11            work->entry = entry;
12            spin_lock_irqsave(&dev->pending_lock, flags);
13            list_add_tail(&work->list, &dev->pending_writes);
14            spin_unlock_irqrestore(&dev->pending_lock, flags);
15        } else {
16            loff_t pos = entry->sector * KERNEL_SECTOR_SIZE;
17            kernel_write(dev->storage_file, entry->data, entry->num_sectors * KERNEL_SECTOR_SIZE,
18                        &pos);
19            atomic_long_inc(&dev->write_count);
20            kfree(entry->data);
21            kfree(entry);
22        }
23    }
24 }
25
26 static void flush_cache(struct smart_block_dev *dev) {
27     unsigned long flags;
28     spin_lock_irqsave(&dev->lock, flags);
29     __flush_cache_unsafe(dev);
30     spin_unlock_irqrestore(&dev->lock, flags);
31     queue_work(dev->wq, &dev->flush_work);
32 }

```

Listing 9: Kernel code snippet from hello/hello.h

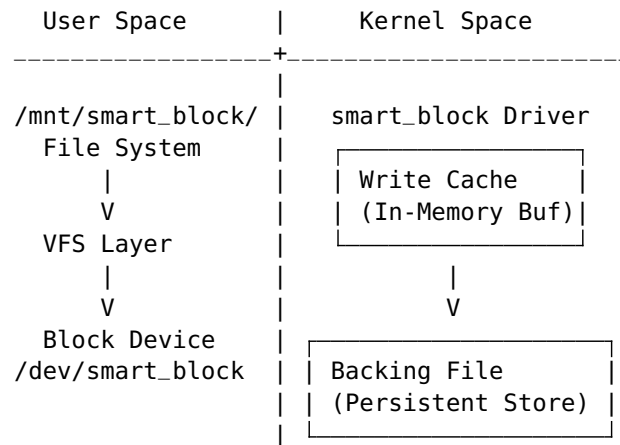
2.7 Interaction with the device through ProcFS

We are maintaining three procfs file to interact with our device

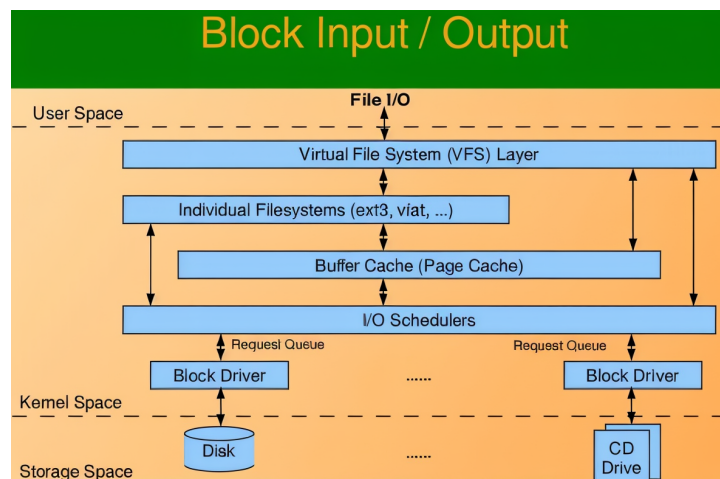
1. The first one is `/proc/smart_block/stats` which tells about Cache size Upperbound(in bytes), Current cache utilization (in bytes) and Number of writes done to disk till now.
2. The second is `/proc/smart_block/flush` if we write "flush" to it then it will flush the cache on the device
3. The third is `/proc/smart_block/cache_size` if you write a number in bytes it will be set as new upper limit on cache size

3 Testing & Conclusion

The testing involves the essential process of setting up a loop device and mounting a file-system on it. Where a loop device is a special mechanism in Linux that allows regular files to be used as block devices, effectively creating a virtual disk from a file. In the initialization steps, the test begins by creating a 2MB backing file using `dd`, which serves as the physical storage space then it uses the command `losetup -fP -show /tmp/smart_block_backing` which associates the backing file with the first available loop device and displays which device was selected. This loop device can then be treated like any other block device in the system, allowing it to be formatted with a filesystem using `mkfs.ext4` and subsequently mounted to a directory with the `mount` command.



The above block diagram illustrates the architectural layers involved in this process. When a filesystem is mounted at `/mnt/smart_block`, it becomes accessible to user space applications through the VFS (Virtual File System) layer, which provides a unified interface for all filesystems. The VFS forwards operations to the `/dev/smart_block` block device, which is handled by the `smart_block` driver in kernel space.



Also note when we are writing data (in test-cases) using `dd` with the `oflag=direct` option, as in the sample command `dd of=$TEST_FILE bs=4096 oflag=direct`, the data is written using direct I/O. This instructs the operating system to bypass the kernel’s internal buffer cache (also known as the page cache), which is depicted as the “Buffer Cache (Page Cache)” layer in the attached image. By default, data written to files passes through this buffer cache, allowing for caching and delayed writes to the underlying storage. However, with `oflag=direct`, the data is transferred directly between user space and the block device, skipping this intermediate caching layer. Also note that this bypass only affects the kernel’s buffer cache and does not interfere with any caching mechanisms implemented by the device driver itself or the storage hardware.

4 Resource Links

The following resources were used to understand and implement our block driver:

1. <https://yannik520.github.io/blkdevarch.html>
2. <https://github.com/rgolubtsov/virtblkiosim/tree/master>